Exercise: Databases

Explore practical techniques for extracting specific customer and product data from MySQL and PostgreSQL databases using Python. Understand how to connect to databases, execute SQL queries, filter data, and export results as CSV files. This lesson helps you apply data extraction skills essential for effective ETL pipelines.

Example

Suppose we're data analysts working for a retail company and our task is to extract customer and product-related data from two relational databases: MySQL and PostgreSQL. The customer data is stored in MySQL, while the product data is stored in PostgreSQL. The company is interested in focusing its marketing budget on marketing the most expensive products to customers from the USA.

We've decided to use Python to connect to both databases, extract the required data, and export it as CSV files to accomplish the task.

> **Note:** Fill in the missing code in the highlighted rows in the code widgets to complete the tasks.

Extracting customer data from MySQL

The first task is to extract data from the `customers` table from MySQL. The marketing department is interested in marketing to clients from the USA. S first task is to extract customers whose `country` is equal to `'USA'`.

Ask

main.py Python 3.10.4

```
import pymysql

# Connect to the database
conn = pymysql.connect(
  host='localhost', port = 3306, database='classicmodels',
  user='root', password='root')

# Create a cursor
cursor = conn.cursor()

# Execute a SQL query
cursor.execute("<<InsertHere>>")
```

Run

Extracting customer data from MySQL

Export customer data to CSV

Next, export the query results as a CSV file for later analysis. Use Python's library `csv` to create an empty CSV file called `USA_customers.csv` and append the query results to the file.

main.py Python 3.10.4

```
# Create a new file and write the results into it
with open('USA_customers.csv', 'w') as write_file:
    writer = csv.writer(write_file, delimiter='|')
    <<insertHere>>
    <<insertHere>>

# Close the cursor and connection
cursor.close()
conn.close()
```

Ask

Run

Exporting customer data

Extract product data from PostgreSQL

After extracting the customer's data from MySQL, we need to extract product data from the `product` table. The company is interested only in marketing the most expensive products. Therefore, we need to extract and export all products with a `buyPrice` greater than `60`.

Connect to PostgreSQL using the `psycopg2` driver. The connection details of the PostgreSQL server are as follows:

- `host = 'localhost'`
- `port = 5432`
- `database = 'postgres'`
- `user = 'postgres'`
- `password = 'postgres'`

main.py Python 3.10.4

```
import psycopg2

# Connect to the database
conn = <<InsertHere>>

# Encode output as UTF8
conn.set_client_encoding('UTF8')

# Create a cursor
cursor = conn.cursor()

# Execute a SQL query
cursor.execute("<<InsertHere>>")
```

Ask

Extracting product data from PostgreSQL

Export the product as CSV

Finally, use the `csv` library to export the query results as CSV for later analysis.

main.py Python 3.10.4

```
<<InsertHere>>

# Create a new file and write the results into it
with open('expensive_products.csv', 'w') as write_file:
  writer = csv.writer(write_file, delimiter='|')
  <<InsertHere>>
  <<InsertHere>>

# Close the cursor and connection
<<InsertHere>>
<<InsertHere>>
```

Run

Exporting product data

Here are the solution codes to the above problems.

Solution code for extracting customer data from MySQL

main.py Python 3.10.4

```
import pymysql

# Connect to the database
conn = pymysql.connect(
  host='localhost', port = 3306, database='classicmodels',
  user='root', password='root')

# Create a cursor
cursor = conn.cursor()
```

Ask

```
# Execute a SQL query
cursor.execute("SELECT * FROM customers WHERE country = 'USA'; ")
```

Run

Solution code: Extracting customer data from MySQL

- **Lines 4–6:** We connect to an active MySQL database.
- **Line 9:** We create a cursor object.
- **Line 12:** We query all the customers from the USA.
- **Line 15:** We fetch the results.
- **Lines 18–19:** We print the results.

Solution code for exporting customer data to CSV

main.py Python 3.10.4

```
# Create a new file and write the results into it
with open('USA_customers.csv', 'w') as write_file:
    writer = csv.writer(write_file, delimiter='|')
    writer.writerows(results)
    write_file.close()

# Close the cursor and connection
cursor.close()
conn.close()
```

Run

Solution code: Exporting customer data

- **Lines 2–5:** We write the query results to `USA_customers.csv`
- **Lines 8–9:** We close the cursor object and connection to the database.

Ask

Solution code for extract product data from PostgreSQL

main.py Python 3.10.4

```
import psycopg2

# Connect to the database
conn = psycopg2.connect(host='localhost', port = 5432, database='postgres
      user='postgres', password='postgres')

# Encode output as UTF8
conn.set_client_encoding('UTF8')

# Create a cursor
cursor = conn.cursor()

# Execute a SQL query
```

Run

Solution code: Extract product data from PostgreSQL

- **Lines 4–5:** We connect to PostgreSQL using `psycopg2`.
- **Line 8:** We encode query output as UTF-8.
- **Line 11:** We create a cursor object.
- **Line 14:** We extract products from the `products` table with a price higher than `60`.
- **Lines 20–21:** We print the query results.
- **Lines 24–25:** We close the cursor object and connection to the database.

Solution code for exporting the product as CSV

main.py Python 3.10 Ask

```
import csv

# Create a new file and write the results into it
```

```
with open('expensive_products.csv', 'w') as write_file:
  writer = csv.writer(write_file, delimiter='|')
  writer.writerows(results)
  write_file.close()

# Close the cursor and connection
cursor.close()
conn.close()
```

Run

Solution code: Export product data

- **Line 1:** We import the `csv` library.
- **Lines 4–7:** We export query results into the CSV file `expensive_products.csv`.
- **Lines 10–11:** We close the cursor object and connection to the database.

Mark As Completed

Ask